



ROBOT SYSTEM DESIGN
LABORATORY

人協働マニピュレーション実装講座(ROS2)

Vol.1

講座の全体的な流れ

- ROS基本知識(前半)
- 休憩は10分~15分
(14:30/16:00)
- 人協働マニピュレーション(後半)



アジェンダ

1. この講座でのゴール

本講座で身につける知識と到達点を整理

2. ROSについて

ロボット開発におけるROSの位置付けと役割

3. ROS2について

特徴・アーキテクチャ・主要ツール（Rviz2等）の概要

4. ROS1との違い

通信方式・ビルドシステム等の比較と移行のポイント

5. ROS2 環境構築

Docker環境・ワークスペースのセットアップ手順

6. ROS2 通信モデル実装

Topic / Service / Action 3方式の実装とサンプル動作確認



この講座のゴール

- ROS2の特徴とアーキテクチャを理解し、今後のロボット開発にROS2を活用するための基礎知識を身につける。
- ROS2とROS1の違いを整理し、既存システムへの移行や新規開発が行えるようにする。
- ROS2の基本的な通信モデルを理解して、実装・動作確認ができるようになっている。



ROSについて

- ROSとは、**Robot Operating System**の略で、OSと付いているが、実はLinuxなどの上で動くミドルウェア。
- ROSは研究・教育・プロトタイプ開発で広く使用されている。
- 2007年に発表され、センサ・モータなどの部品をプログラムでつなぐ通信機能や、便利なライブラリを多数提供されています。

ROS



ROSについて

ROSはロボット開発のために必要な、以下の機能で成り立っている。

plumbing (通信)

ROSの中核で、ノード間のデータ交換を担う通信基盤。
Topic (Pub/Sub)、Service、Actionなどにより分散処理を実現。
異なるプロセス・マシン間でも柔軟かつ非同期に連携できる。



tools (ツール群)

開発・デバッグ・可視化を支援するための各種ツール。
例としてrviz (可視化) やros2 topicコマンドなどがある。
システムの状態確認や効率的な開発を可能にする。

capabilities (機能群)

ロボット開発に必要な基本機能をパッケージとして提供。
ナビゲーション、マニピュレーション、認識などが含まれる。
これらを組み合わせることで高度なロボット機能を実現する。

community (コミュニティ)

OSSとしてのコミュニティと再利用可能な資産の集合。
多数のパッケージやドキュメント、ユーザー知見が共有されている。
開発効率を高め、他者の成果を活用できる環境を形成している。

ROSについて

ロボット開発は多様な選択肢があり、用途や要求に応じて使い分けられる。

Vendor SDK（メーカー独自フレームワーク）

Universal Robots URScript、FANUC Karel、ABB RAPID、KUKA KRLなど産業用ロボットの専用言語。
メーカー保証・リアルタイム性に優れる一方、汎用性や他社連携に制限がある。

OROCOS / YARP（研究系ミドルウェア）

OROCOSはOpen Robot Control Softwareの略。リアルタイム制御指向で精密な動作制御に強い。
YARPはヒューマノイドロボット（iCub等）の研究で広く利用されている通信基盤。

Isaac SDK / Isaac ROS（NVIDIA系プラットフォーム）

GPU・AI処理に最適化されたロボティクス開発環境。
シミュレーション（Isaac Sim）と認識・学習機能を統合的に提供。

Embedded / MATLAB（組み込み・モデルベース開発）

C/C++によるマイコン直接実装。リソース制約の厳しい小型ロボットで採用。
MATLAB/Simulinkによるモデルベース開発で制御設計から実装まで一貫して行える。



ROSについて

- ROSがデファクトスタンダードになった理由

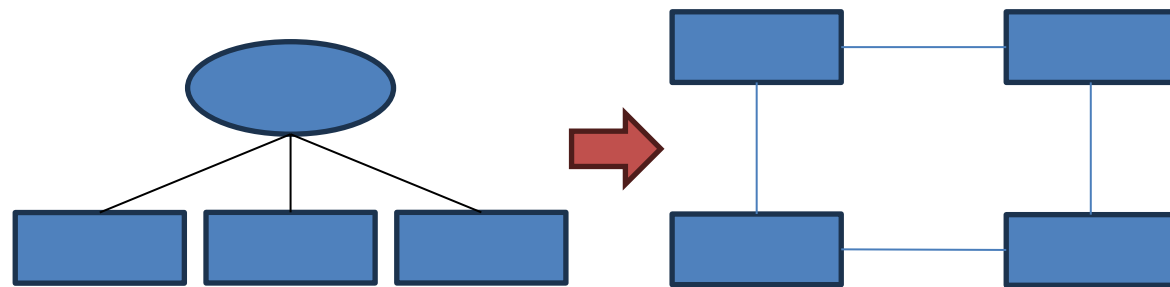
1. 既存の資産を使用して容易かつ迅速にロボットを構築できる
2. 試行やデバッグを容易にするツールが提供されている
3. 上記のような開発資産の共有と進化を促進する枠組みを提供している

- ROS2が必要になった背景

従来のROS（ROS1）が、実用ロボットに求められる要件を満たしきれなくなったため、ROS2が必要になった。
研究用から実用・商用ロボットへスケール

要件:

1. 通信の信頼性・リアルタイム性の不足
2. 分散システム・大規模化への限界
3. 実運用（プロダクション）への対応不足
4. マルチプラットフォーム対応の弱さ



ROS2について



The Robot Operating System (ROS) is a set of software libraries and tools for building robot applications. From drivers and state-of-the-art algorithms to powerful developer tools, ROS has the open source tools you need for your next robotics project.

(引用: <https://docs.ros.org/en/jazzy/index.html>)

ROS2(Robot Operating System 2)は、ロボットアプリケーション開発のための、ソフトウェアライブラリとツール群のこと。(→ミドルウェア)

ドライバや最先端のアルゴリズムから強力な開発ツールに至るまで、ROSにはロボットプロジェクトに必要なオープンソースツールが揃っている。

- ROS1の後継ミドルウェア。
- DDS(Data Distribution Service)を通信基盤として採用している。
- **分散システム・マルチロボット・リアルタイム性を強化。**
- **自律移動・マニピュレーション・センサ処理・位置推定**などの機能がすぐ使える形で提供。

DDSとは：分散システムにおける“データ通信の仕組み(ミドルウェア規格)”です。リアルタイムかつ信頼性の高いデータ共有のための国際標準通信規格です。主に、Pub/Sub通信モデルを使用し、遅延が少ない点と機器間の柔軟な接続性から多くの産業システムで採用されている。

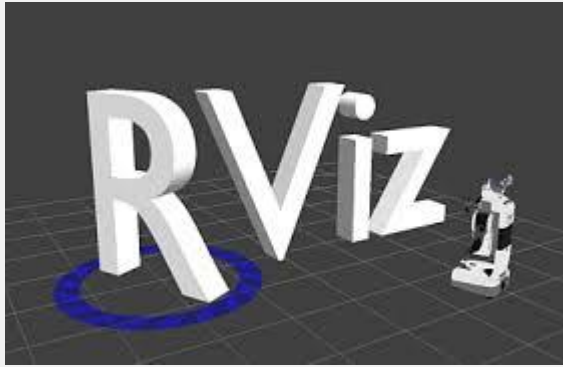
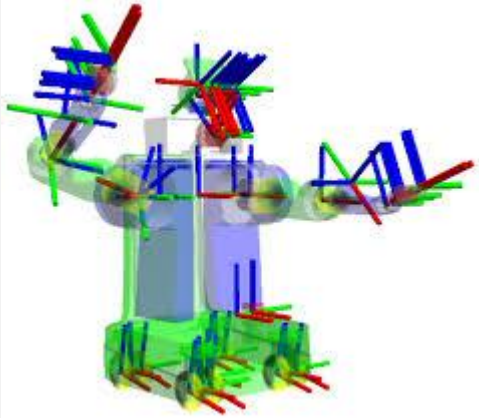


ROS2について

ROS2 バージョン	Foxy LTS	Humble LTS	Jazzy LTS	Lyrical LTS
Ubuntuバージョン	20.04 LTS	22.04 LTS	24.04 LTS	26.04 LTS
リリース日	2020年5月	2022年5月	2024年5月	2026年5月
サポート期間	2023年5月	2027年5月	2029年5月	2031年 5 月
バージョン イメージ画像				

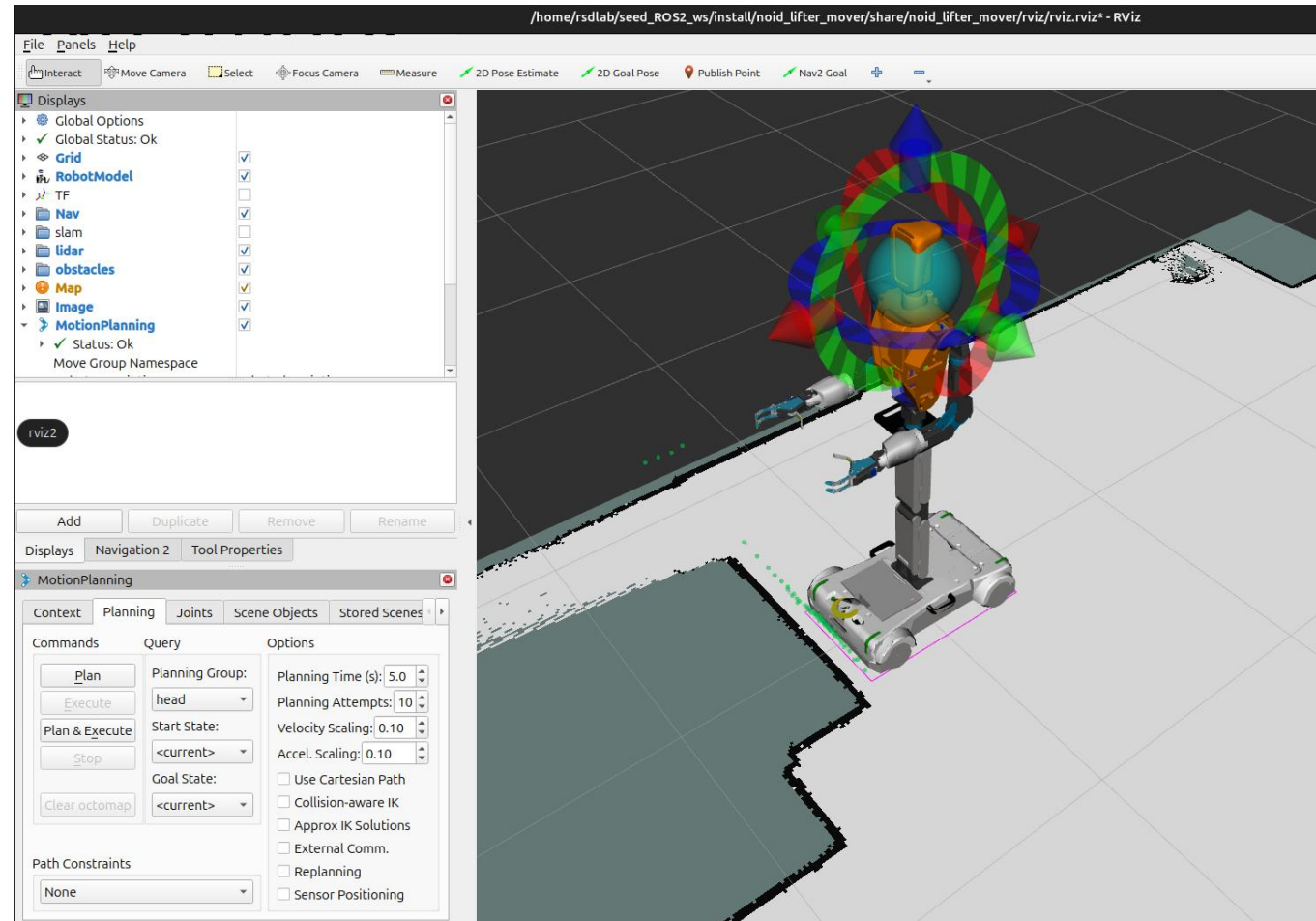
LTS(Long Term Support): 最大5年間の公式サポートが提供される
非LTS: およそ1.5~2年程度のサポート

ROS2 主要ツール

Movelt2	Gazebo	RViz2	TF2
モーシヨンプランニング アームの動作計画・逆運動学・衝突回避を自動化	3D物理シミュレーター 実機不要で物理演算・センサーをシミュレーション	3D可視化ツール センサー・TF・ロボットモデルをリアルタイム表示	座標変換ライブラリ 各パーツ・センサー間の座標フレームを管理
			

Movelt2

関節の角度などを設定して、RViz2上で実行し確認することも可能



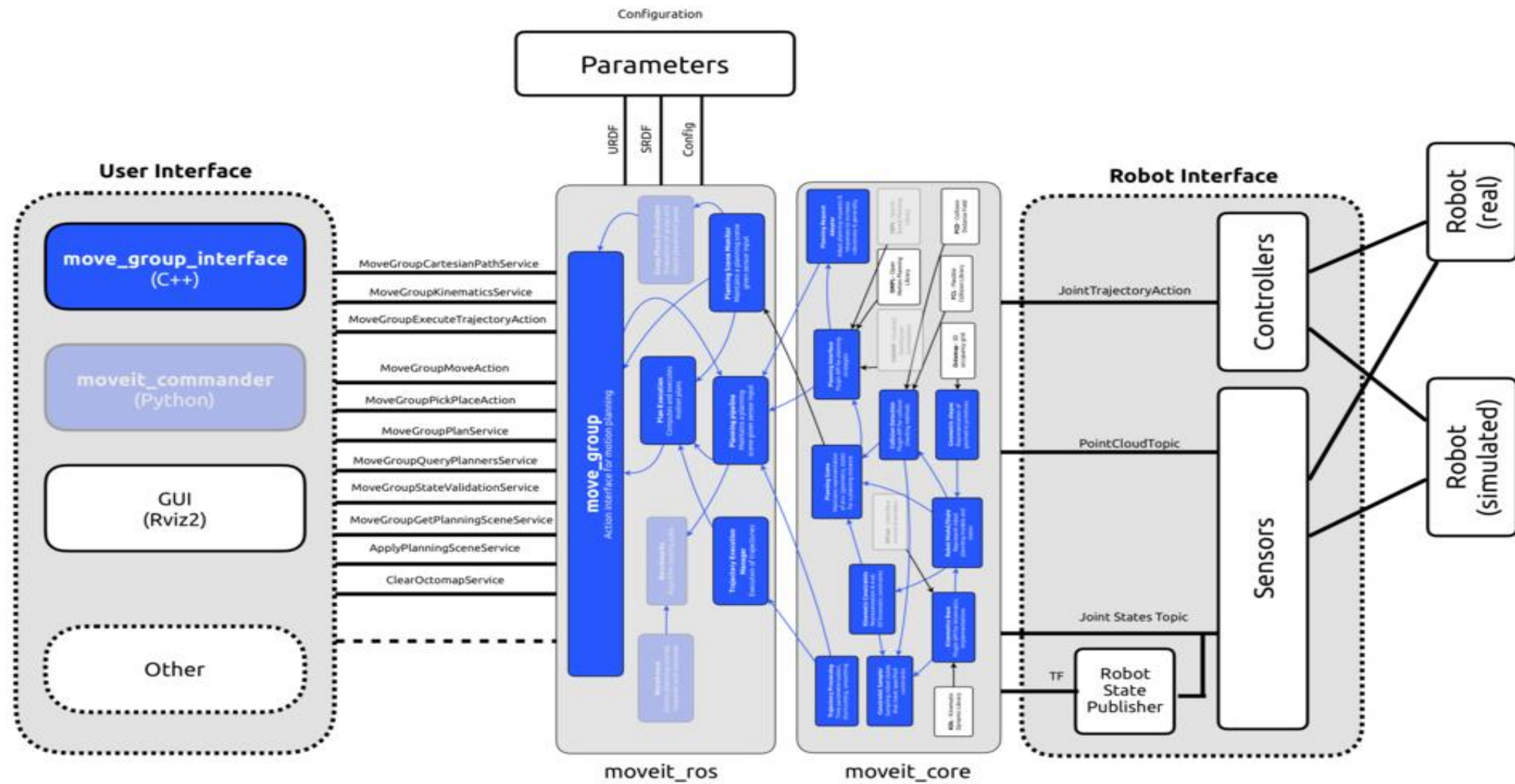
MoveIt2

項目	ROS1 MoveIt	ROS2 MoveIt2
パッケージ名	moveit_ros	moveit2
ノード通信	ROS Masterが必要	DDS(ノード間直接通信)
コントローラ	ros_control	ros2_control
Python API	moveit_commander	moveit_py
リアルタイム	非対応	RTOS対応可能

ROS2 MoveIt2への移行はAPIの基本思想を維持しつつ設計が刷新されている

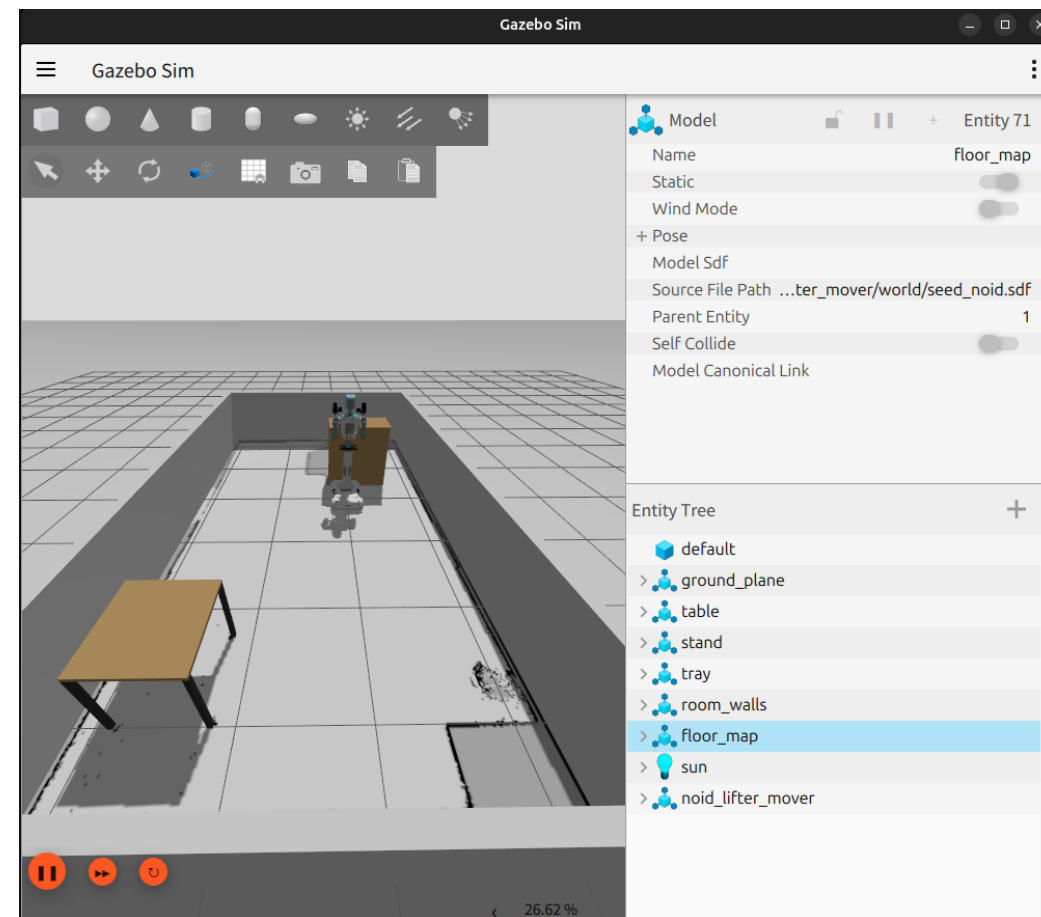


MoveIt2



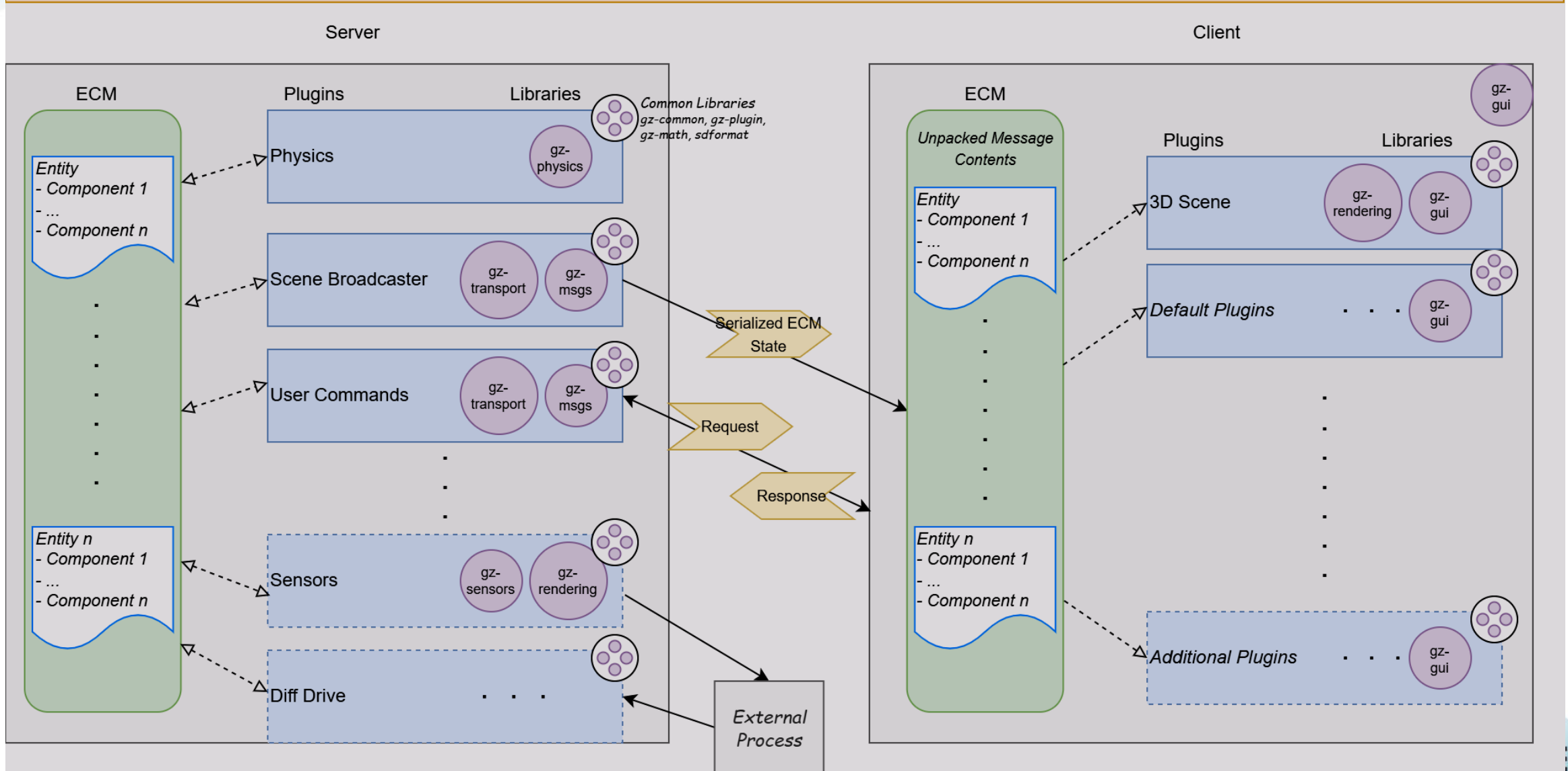
Gazebo Sim

- 物理シミュレーションを行う場合は、Gazeboを使用する。Rvizは「データの可視化」であるのに対し、Gazeboは「ロボット・センサ・環境を物理エンジンで再現」する役割を担う。両者は併用されることが多い。



Gazebo Sim

Gazebo Sim Architecture



Rviz2

- Rvizはシミュレーターではなく、データを可視化するツール。ROS2のトピックを購読してリアルタイムで可視化できる。

センサーデータの可視化	ロボット・空間の可視化	ナビゲーション・経路の可視化	デバッグ・カスタム表示
LiDAR・レーダースキャン カメラ画像 IMU/その他センサー	ロボットモデル 地図 TFアーム	経路・軌跡 位置推定・オドメトリ 目標地点の指定	Maker(任意描画) 設定の保存・共有 カスタムプラグイン



座標系の管理

ツリー構造

ロボットの各部位の親子関係を管理

静的・動的変換

固定部位と稼働部位の両方に対応

情報の配信と取得

配信(Broadcast)

現在の位置関係をシステム全体に共有

取得(Listen)

任意の時刻・座標間の変換を計算

ROS1との違い

項目	ROS1	ROS2
ビルドシステム	catkin	ament + colcon
通信アーキテクチャ	マスターによるノード管理(中央集中型)	DDSベースの分散型アーキテクチャ(マスター不要)
リアルタイム性	基本的には非対応	リアルタイム制御を考慮した設計で対応可能
QoS (Quality of Service) 設定	原則なし	速度や信頼性などを設定可能
セキュリティの強度	標準では低い	高い(暗号化、認証、アクセス制御など)
対象OS	Linux(主にUbuntu)	Linux・Windows・macOSなど
ネットワーク品質	ネットワーク品質を前提に設計されていて制御できない	ネットワーク品質が変動する環境でも動くように設計され、制御できる
マルチロボット対応	単体での利用が中心	分散設計により複数台(マルチロボット対応)
応用範囲	研究、教育向け	産業用途まで幅広く応用可能
将来性	低い(EOL済み)	高い(継続開発、機能拡張、長期サポート、セキュリティ対策など)



ROBOT SYSTEM DESIGN

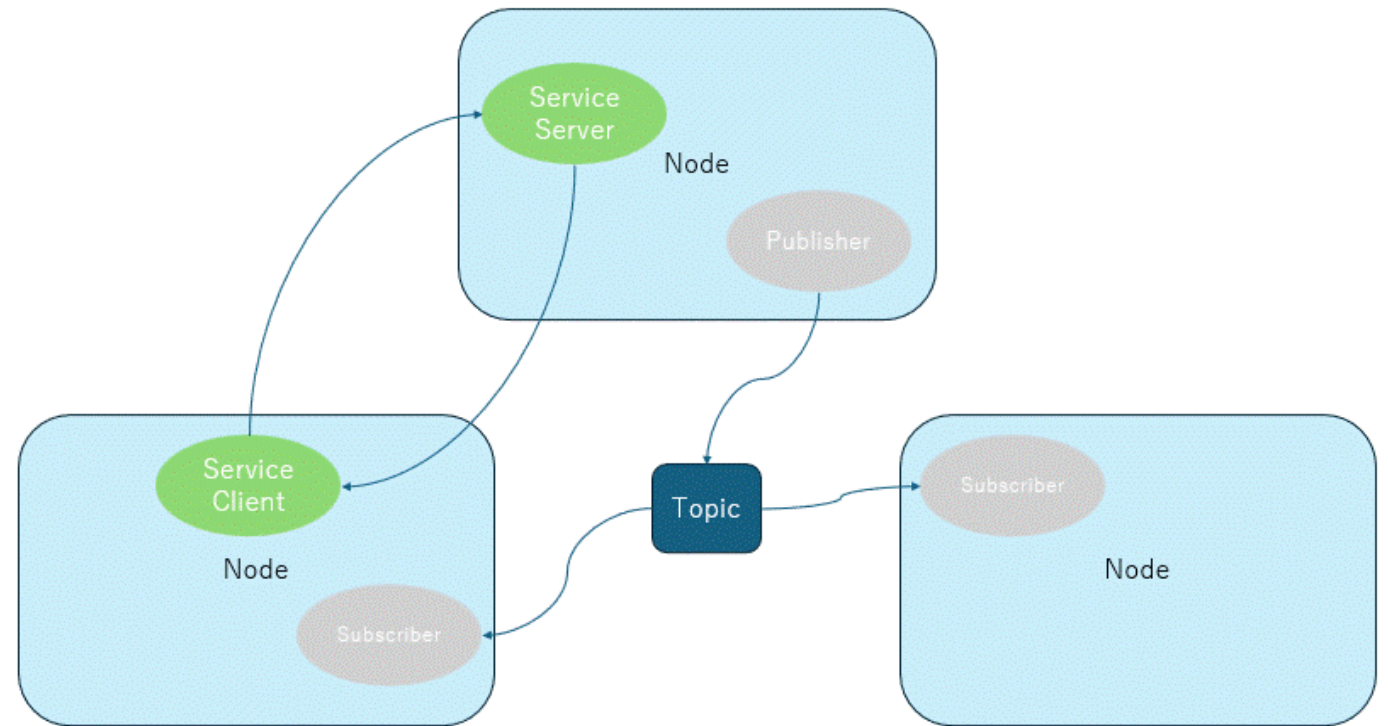
LABORATORY

ROS2 通信モデル

ROS2 通信モデル

ROS2には3つの通信インターフェースと2つのノード機能・設定があります。

1. Topic通信(Pub/Sub)
 2. Service通信(Request/Response)
 3. Action通信(長時間処理)
4. Parameters(パラメータ)
 5. QoS(Quality of Service)



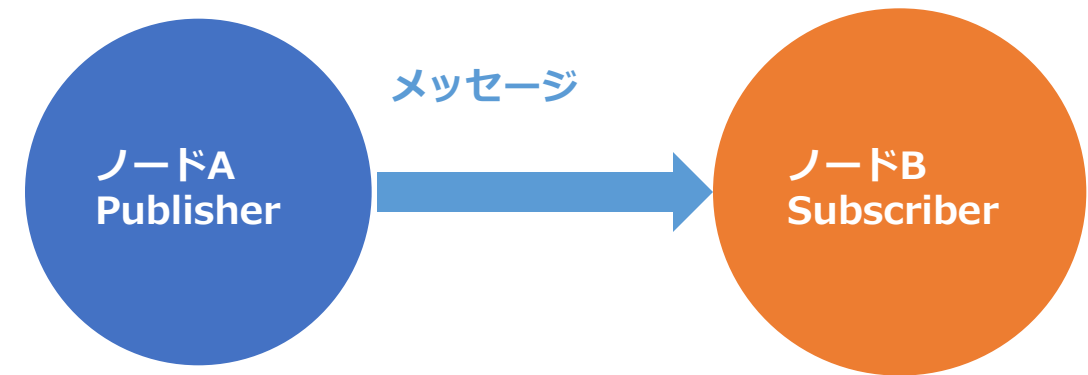
ROS2 通信モデル

ROS2では各機能を独立した**ノード**として実装し、**ノード間通信**でデータをやり取りします。

■ ノード間通信の特徴

- ・ 各ノードは**独立したプロセス**として動作
- ・ ノードの追加・削除が容易（疎結合）
- ・ 異なる言語・PC・プラットフォーム間でも通信可能
- ・ 用途に応じて**Topic / Service / Action**を使い分け

ノード間通信のイメージ



通信方式

Topic
Pub/Sub

Service
Req/Res

Action
長時間処理

ROS2 通信モデル

Topic通信

ノード間通信の基本

Topic : データの通り道

message : 実際を送るデータ

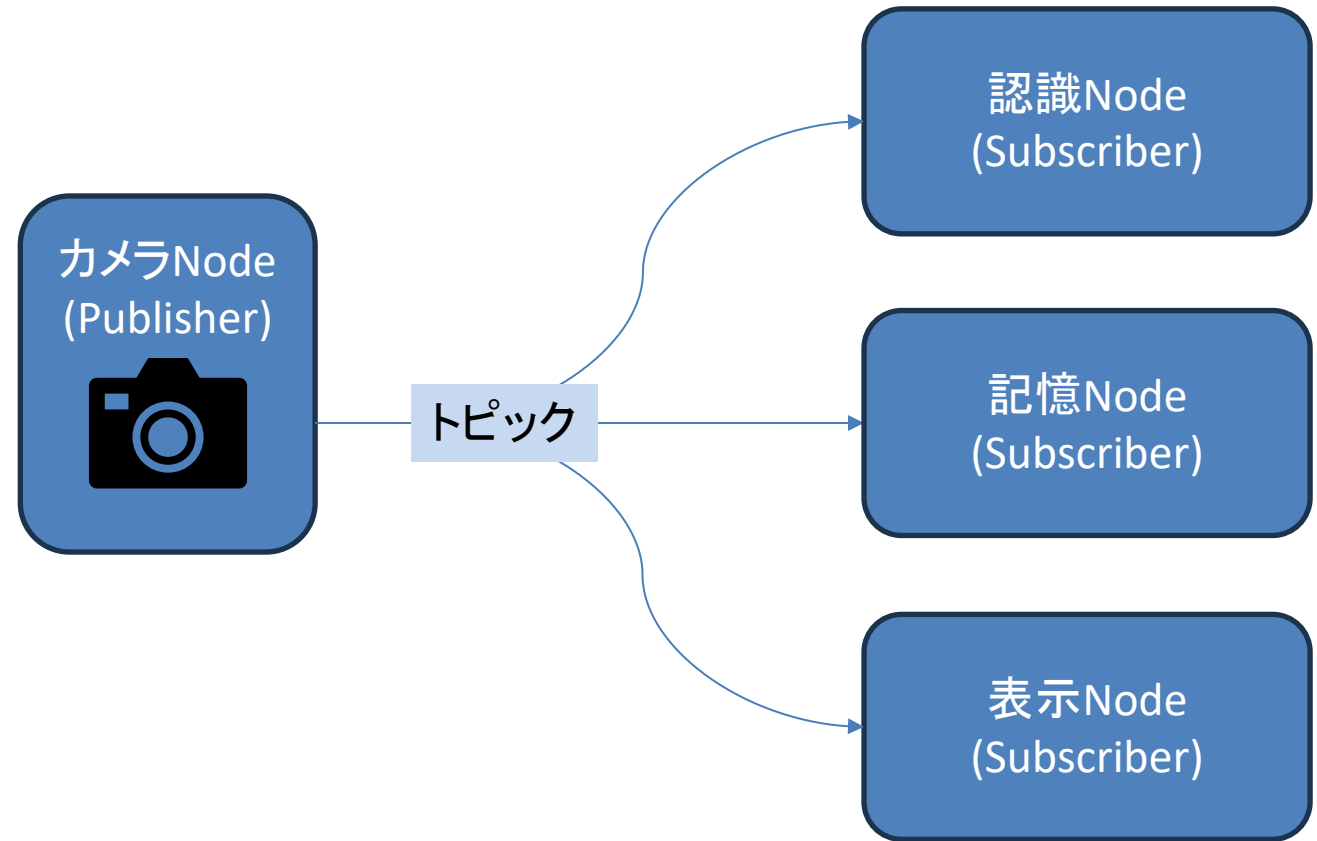
特徴 :

- ・ 非同期通信
- ・ 1対多/多対1が可能
- ・ ストリーム型

例 :

画像データ : /camera/image_raw

速度指令 : /cmd_vel



ROS2 通信モデル

Service通信

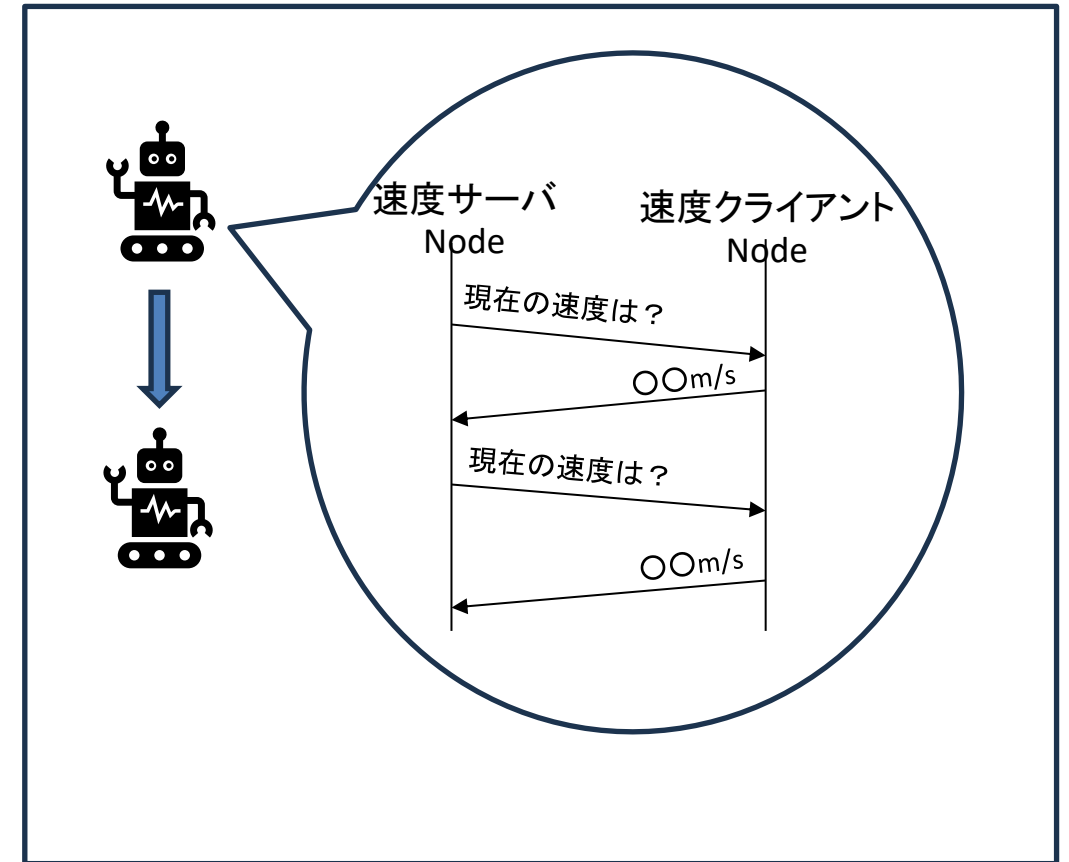
- Request/Response: データを送る

特徴:

- 同期通信(呼び出し -> 応答待ち)
- 1対1
- 単発処理
- 成功・失敗を明確に返す

例:

ロボットの速度取得
モード切替



ROS2 通信モデル

Action通信

長時間処理 + 途中経過が必要な場合

特徴：

- ・ 非同期
- ・ フィードバックあり
- ・ キャンセル可能

例：

ナビゲーション
アーム操作



ROS2 通信モデル

Parameter

ノードの設定値を管理する仕組み
実行中に値の変更が可能

特徴：

ノード単位で保持
動的変更が可能

例：

制御ゲイン
動作モード
閾値設定

ros2 param list	パラメータ名の一覧表示
ros2 param get	パラメータの値と型の取得
ros2 param set	パラメータの値変更

パラメータ宣言(パラメータ名,デフォルト値)

```
self.declare_parameter('max_speed', 1.5)
```

ノード起動時に”-ros-args -p max_speed:=2.0”
でデフォルト値に上書きできる



ROS2 通信モデル

QoS(Quality of Service)

通信品質を制御する仕組み

特徴：

ネットワーク状況によって最適化可能
ROS1にはなかった重要機能

例：

カメラ：BEST Effort(遅延重視)
制御信号：RELIABLE(確実性重視)

主な設定：

Reliability (信頼性)

- RELIABLE / BEST Effort

History (履歴)

- KEEP_LAST / KEEP_ALL

Depth (バッファサイズ)

Durability (データ保持)



ROBOT SYSTEM DESIGN

L A B O R A T O R Y

ROS2 環境構築

ROS2環境構築

- OS:
Linux (Description: Ubuntu 24.04.4 LTS)
- ミドルウェア
ROS2 Jazzy Jalisco
- 使用プログラミング言語
Python / C++
- 実装環境
SEED-R7シリーズ(THK社)



ROS2環境構築

- ロケール設定

```
$ sudo apt update && sudo apt upgrade
```

```
$ sudo apt install locales
```

```
$ sudo locale-gen ja_JP ja_JP.UTF-8
```

```
$ sudo update-locale LC_ALL=ja_JP.UTF-8 LANG=ja_JP.UTF-8
```

```
$ export LANG=ja_JP.UTF-8
```

確認

```
$ locale
```

```
LANG=ja_JP.UTF-8
```

...

- 必要ツールインストール

```
$ sudo apt install -y curl gnupg lsb-release python3-colcon-common-extensions  
python3-rosdep python3-vcstool git software-properties-common python3-pip python3-  
colcon-clean
```



ROS2環境構築

- ROS2 aptリポジトリ追加

```
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

- ROS2 Jazzyインストール

```
$ sudo apt update
```

```
$ sudo apt install ros-jazzy-desktop
```

```
$ sudo apt install -y ros-jazzy-rqt-graph ros-jazzy-ros2-controllers ros-jazzy-control-test-assets ros-jazzy-ros2-control ros-jazzy-ros-gz ros-jazzy-gz-ros2-control
```



ROS2環境構築

- 環境設定(起動時に読み込まれるように設定)

```
$ echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc  
$ source ~/.bashrc
```

- rosdep初期化

```
$ sudo rosdep init  
$ rosdep update
```

- パッケージインストール

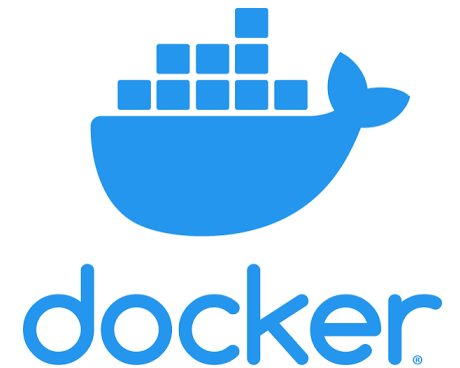
```
$ sudo apt install ros-${ROS_DISTRO}-nav2-bringup ¥  
ros-${ROS_DISTRO}-laser-proc ¥  
ros-${ROS_DISTRO}-laser-filters ¥  
ros-${ROS_DISTRO}-ros2-control-cmake ¥  
ros-${ROS_DISTRO}-tf-transformations ¥  
ros-${ROS_DISTRO}-moveit ¥  
ros-${ROS_DISTRO}-moveit-ros-planning-interface ¥  
ros-${ROS_DISTRO}-moveit-core ¥  
ros-${ROS_DISTRO}-moveit-common ¥  
ros-${ROS_DISTRO}-moveit-ros-planning ¥  
ros-${ROS_DISTRO}-moveit-visual-tools ¥  
mplayer ¥  
ros-${ROS_DISTRO}-joy-linux
```



ROS2環境構築(Docker)

- Dockerとは
アプリケーションとその実行に必要な環境(ライブラリ・設定・依存関係)を一つの**コンテナ**にまとめて、どこでも同じように動かせるようにするプラットフォーム。

Docker Desktopを起動してください



ROS2環境構築(Docker)

- Gazeboを使用してシミュレーションを行う為、メモリ・プロセッサをWSL2 で使用できるように設定する

C:¥User¥ユーザ名¥.wslconfig

ホストPCのスペックを確認して設定

メモリ: 32GB -> 24GB

プロセッサ: 8core -> 6core

タスクマネージャーからホストPCのスペックは分かります。

設定が完了したらPowershellから以下のコマンドを実行

> wsl --shutdown

```
[wsl2]
memory=24GB
processors=6
swap=8GB
gpuSupport=true
```



ROS2環境構築(Docker)

- WSL2内にROS2のDocker環境をクローン・ビルド

```
$ git clone -b ros2-jazzy https://github.com/rsdlab/docker-ros2-desktop-vnc.git
```

```
$ cd docker-ros2-desktop-vnc/jazzy
```

```
$ docker build -t tiryoh/ros2-desktop-vnc:jazzy .
```

- Docker Desktopの起動・Dockerコンテナを起動

```
$ docker run -p 6080:80 --shm-size=512m tiryoh/ros2-desktop-vnc:jazzy
```

or

Docker Desktopの"**Containers**"にビルドされたコンテナの"▷"を押下
コンテナが起動したら<http://127.0.0.1:6080/> にアクセス



ROS2環境構築(Docker)

noVNCが開けて、接続を選択するとUbuntu MATEが開く



ROS2環境構築(Docker)

- noVNCが使用できない場合

<https://www.tightvnc.com/download.php>からtightvncをダウンロード



- DockerImageビルド

```
$ docker build -t tiryoh/ros2-desktop-vnc:jazzy .
```

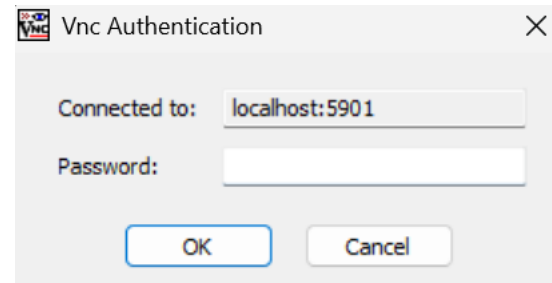
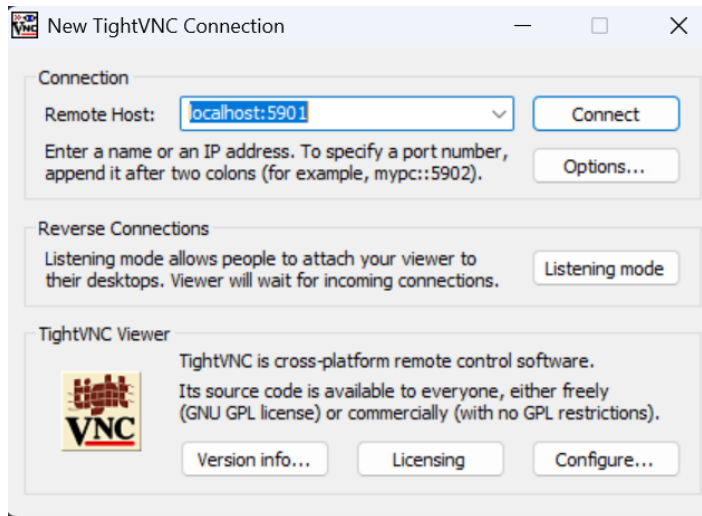
- Dockerコンテナ起動

```
$ docker run -p 6080:80 -p 5901:5901 --shm-size=512m tiryoh/ros2-desktop-vnc:jazzy
```



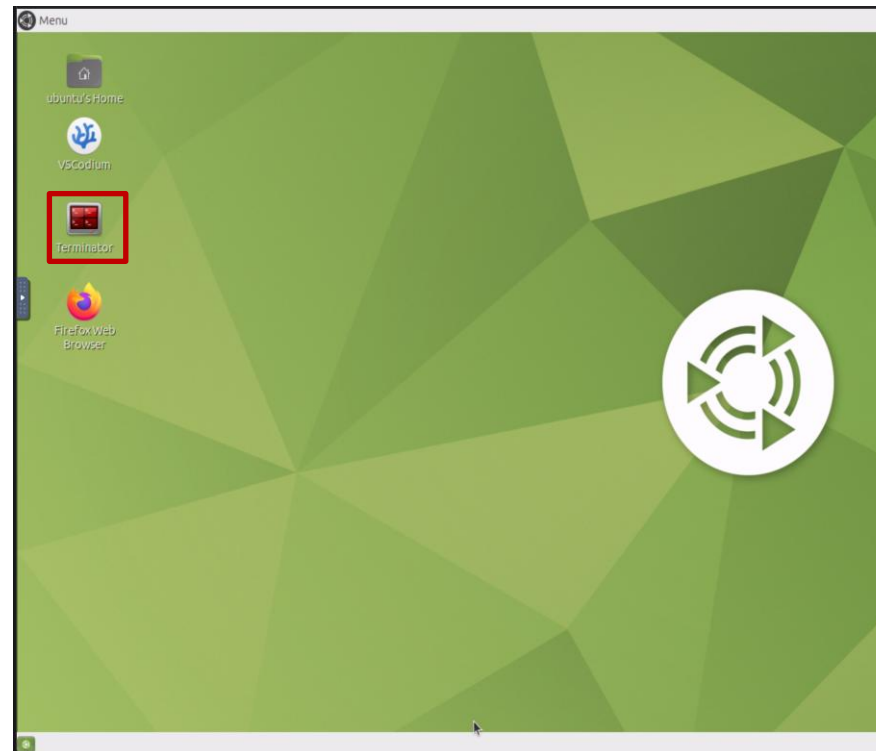
ROS2環境構築(Docker)

Remote Hostにlocalhost:5901を入力して接続する。
パスワードは、“ubuntu”



Ubuntu MATEの操作方法

- Terminator or “Ctrl + Alt + T”でターミナルを開く
- コピーは、“Ctrl + Shift + C”で可能
- ペーストは、“Ctrl + Shift + V”で可能



ROS2環境構築(Docker)

- VSCodeからDocker環境にアタッチ

VSCodeの左のメニューからContainersを選択して起動中のコンテナを右クリックして“Visual Studio Codeをアタッチ”を押下

VSCodeの左下に画像のようになっていたらアタッチできています。

```
>< コンテナ - tiryoh/ros2-desktop-vnc:jazzy (ado...
```



ROS2環境構築

- 動作確認

talker/listenerテスト

(ターミナル1)

```
$ ros2 run demo_nodes_cpp listener
```

(ターミナル2)

```
$ ros2 run demo_nodes_cpp talker
```

```
rsdlab@rsdlab-NUC11TNKi3:~$ ros2 run demo_nodes_cpp listener
```

```
rsdlab@rsdlab-NUC11TNKi3:~$ ros2 run demo_nodes_cpp talker
```



ROS2ビルド方法

- サンプルビルド

```
$ mkdir -p ~/colcon_ws/src
```

```
$ cd ~/colcon_ws/src
```

```
$ git clone https://github.com/ros2/examples.git -b jazzy
```

```
$ cd ~/ros2_ws
```

```
$ colcon build
```

```
$ source install/setup.bash
```



ROS2ビルド方法

別々のターミナルで起動

(ターミナル①)

```
$ ros2 run examples_rclcpp_minimal_subscriber subscriber_lambda
```

(ターミナル②)

```
$ ros2 run examples_rclcpp_minimal_publisher publisher_lambda
```

```
rsdlab@rsdlab-NUC11TNK13:~/ros2_ws$ ros2 run examples_rclcpp_minimal_subscriber subscriber_lambda
```

```
rsdlab@rsdlab-NUC11TNK13:~/ros2_ws$ ros2 run examples_rclcpp_minimal_publisher publisher_lambda
```

(Ctrl + Cで停止できます。)



ROS2ビルド方法

• ディレクトリ構成

```
ros2_ws └─ build  中間生成物(Cmakeのキャッシュ・オブジェクトファイル・コンパイル途中の成果物)
          └─ install 最終成果物(実行ファイル・ライブラリ・package.xml)
          └─ log    ビルド・実行時のログ
          └─ src    ビルドに必要なソースファイル
```

ビルドオプション

- `--packages-select <pkg>` 指定パッケージのみビルド
- `--packages-up-to <pkg>` 指定パッケージとその依存を順にビルド
- `--symlink-install` install先をシンボリックリンク化(再ビルド負荷を軽減)
- `--cmake-args -DCMAKE_BUILD_TYPE=Release` CMakeへ引数を渡す(例: リリースビルド)
- `--parallel-workers N` 並列ビルド数を指定(N=同時実行数)
- `--event-handlers console_direct+` ビルドログをリアルタイム表示





ROBOT SYSTEM DESIGN

LABORATORY

ROS2 通信実装

ROS2 通信実装

- 事前準備

(Linux)

```
$ mkdir -p ~/colcon_ws/src
```

```
$ cd ~/colcon_ws/src
```

```
$ mkdir ros2_topic_sample ros2_service_sample  
ros2_action_sample
```



Topic通信実装

Publisher

`create_publisher(msg_type, topic_name, qos_profile)` : Publisherを作成する

引数	型	説明
<code>msg_type</code>	<code>Type[Msg]</code>	メッセージのクラス自体
<code>topic_name</code>	<code>str</code>	トピック名(/chatter)
<code>qos_profile</code>	<code>QoSProfile</code> <code>int</code>	QoS設定。整数でも可。

`publish(msg)` : メッセージを送信する。

引数	型	説明
<code>msg</code>	<code>MsgT</code>	送信するメッセージのインスタンス

`create_timer(timer_period_sec, callback)`

引数	型	説明
<code>timer_period_sec</code>	<code>float</code>	コールバックを呼ぶ間隔(秒)
<code>callback</code>	<code>Callback[[], None]</code>	定期実行する関数。引数なし・戻り値なし



Topic通信実装

Subscriber

`create_subscription(msg_type, topic_name, callback, qos_profile)` : Subscriberを作成する。

引数	型	説明
<code>msg_type</code>	<code>Type[Msg]</code>	メッセージのクラス自体
<code>topic</code>	<code>str</code>	購読するトピック名
<code>callback</code>	<code>Callback[[MsgT], None]</code>	受信時に呼ばれる関数。
<code>qos_profile</code>	<code>QoSProfile Int</code>	QoS設定



Topic通信実装

- ビルド

(Linux)

```
$ cd ~/colcon_ws
```

```
$ colcon build --packages-select exercise_topic
```

```
$ source install/setup.bash
```

- 実行

(ターミナル①)

```
$ ros2 run ros2_topic_sample listener
```

(ターミナル②)

```
$ ros2 run ros2_topic_sample talker
```



Service通信実装

Server

`create_service(srv_type, srv_name, callback)` : Serviceサーバーを作成する。

引数	型	説明
<code>srv_type</code>	<code>Type[Msg]</code>	サービスのメッセージ型
<code>srv_name</code>	<code>str</code>	サービス名
<code>callback</code>	<code>Callback[[MsgT], None]</code>	コールバック関数。



Service通信実装

Client

`create_client(srv_type, srv_name)` : Serviceクライアントを作成する。

引数	型	説明
<code>srv_type</code>	<code>Type[Msg]</code>	サービスのメッセージ型
<code>srv_name</code>	<code>str</code>	サービス名

`call_async(request)` : リクエストを非同期送信し、Futureを返す。

引数	型	説明
<code>request</code>	<code>Msg.Request</code>	送信するリクエストのインスタンス



Service通信実装

- ビルド

```
$ cd ~/colcon_ws
```

```
$ colcon build --packages-select ros2_service_sample
```

```
$ source install/setup.bash
```

- 実行

(ターミナル①)

```
$ ros2 run ros2_service_sample service_server
```

(ターミナル②)

```
$ ros2 run ros2_service_sample service_client
```



Action通信実装

Server

ActionServer(node, action_type, action_name, execute_callback) : Actionサーバーを作成する。

引数	型	説明
node	Node	アクションを登録するノード
action_type	Type[Action]	アクションのメッセージ型
action_name	str	アクション名
execute_callback	Callback[[GoalHandle], Result]	ゴール実行時のコールバック

Client

ActionClient(node, action_type, action_name) : Actionクライアントを作成する。

send_goal_async(goal) : ゴールを送信し、Futureを返す。

引数	型	説明
node	Node	アクションを利用するノード
action_type	Type[Action]	アクションのメッセージ型
action_name	str	アクション名

Action通信実装

- ビルド

```
$ cd ~/colcon_ws
```

```
$ colcon build --packages-select ros2_action_sample
```

```
$ source install/setup.bash
```

- 実行

(ターミナル①)

```
$ ros2 run ros2_action_sample action_server
```

(ターミナル②)

```
$ ros2 run ros2_action_sample action_client
```



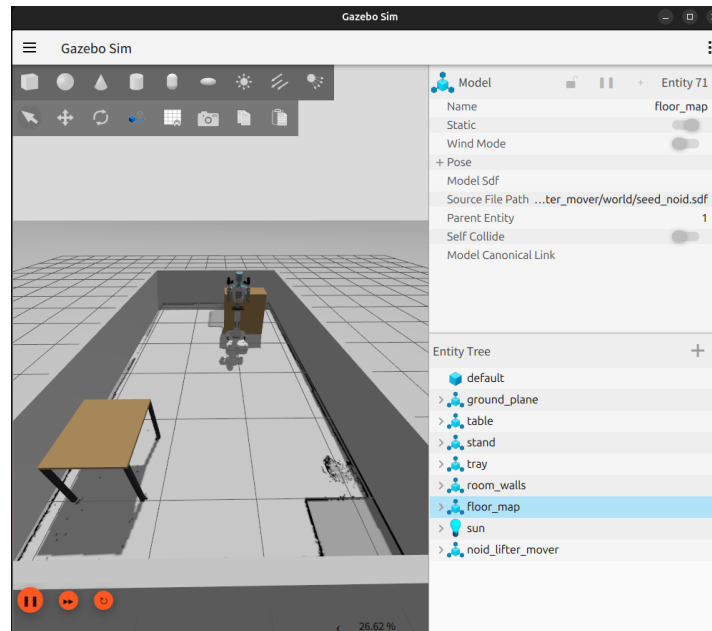
その他ファイルの説明

- package.xml
パッケージのメタ情報(名前・バージョン・メンテナ・ライセンス)と、ビルド時／実行時の依存パッケージをXML形式で宣言するファイル
 - <depend> 依存パッケージ(rclpy, std_msgs など)
 - <buildtool_depend> ビルドツール(ament_python)の指定
- setup.cfg
ament_python向けの設定ファイル。実行ファイル(scripts)のインストール先を指定し、ros2 runからノードを呼び出せるようにする
 - [develop]/[install] script_dir, install_scripts をパッケージ名で指定
- setup.py
Pythonパッケージのビルド・インストール定義ファイル。colcon build時に参照される
 - entry_points={'console_scripts': [...]} ros2 runで起動するノードを登録
 - install_requires Pythonの依存パッケージを宣言



まとめと後半予告

- 今回は、ROS1とROS2の簡単な違い、ROS2の通信モデルの解説・実装を行いました。
- 次回は、SEED-Noïdの環境構築～シミュレーションの動作確認を行います。





ROBOT SYSTEM DESIGN

LABORATORY

参考文献

参考文献

- ROS2 Docs:
<https://docs.ros.org/>
- Gazebo Docs:
<https://gazebo.org/docs/harmonic/getstarted/>
- Nav2 Docs:
<https://docs.nav2.org/>
- MoveIt2:
<https://moveit.picknik.ai/main/index.html>
- ROS2とPythonで作って学ぶAIロボット入門 改訂第2版
(出村・萩原・升谷・タン著, 講談社)

